

CS1010J Lecture #09 Unit 1

Basic OOP I

A paradigm shift



Department of Computer Science
School of Computing

Learning Objectives

- At the end of this lecture, you should understand:
 - ❑ The basic OOP concepts such as **class**, **object**, **attribute**, **constructor**, **member method**, **modifier** and **object reference**.
 - ❑ How to formulate solutions in the object-oriented paradigm.

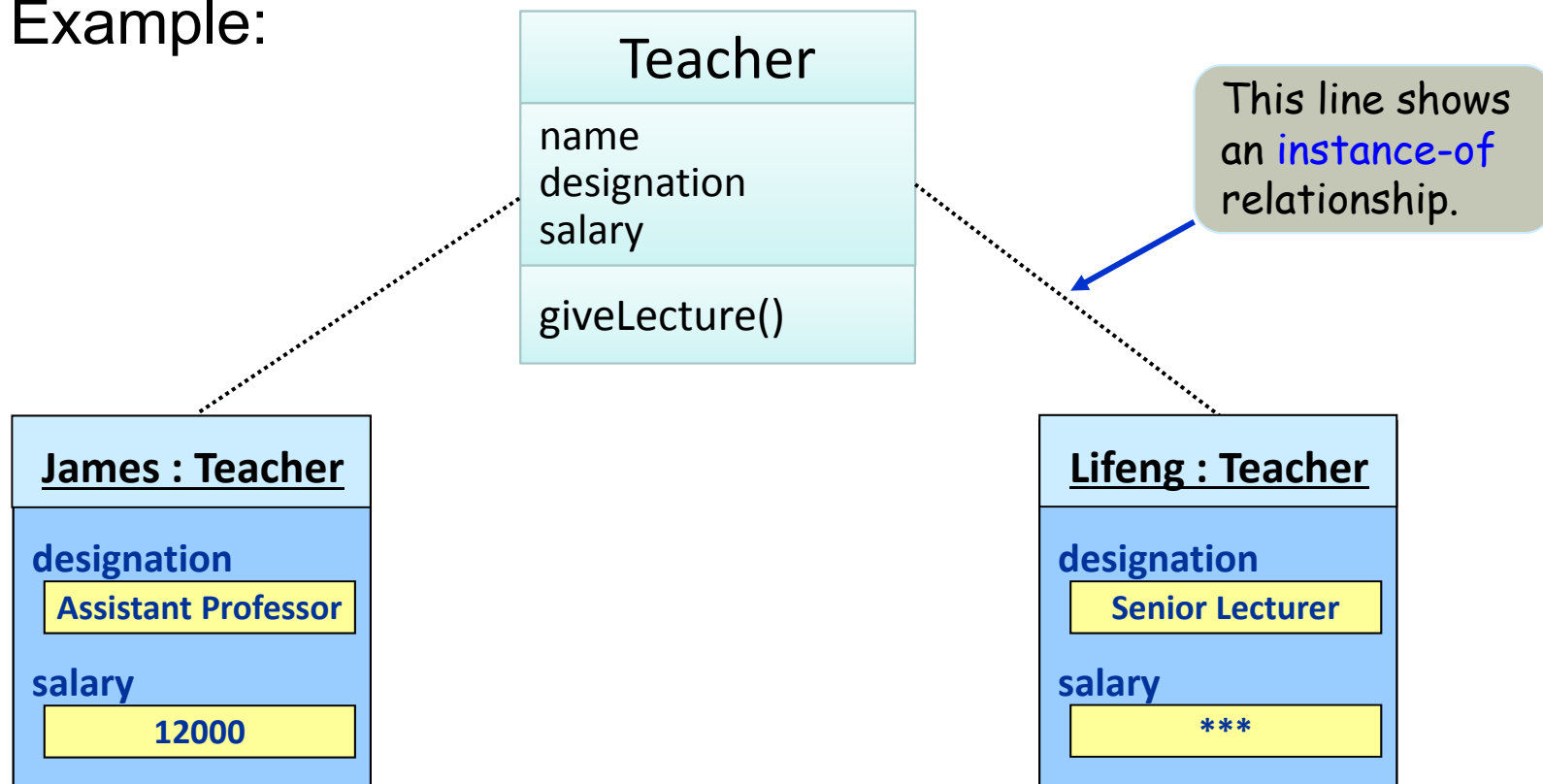


Programming Models

- All the programming exercises we have attempted so far are **computation oriented**.
 - A solution program is composed of a single class containing a main method and some other methods.
 - This is the style of **procedure programming**.
- Today we will learn another way of writing programs, in which we model entities (objects) we identify in a problem domain.
 - Each object has **data** and **behaviours** (functionalities).
 - Computation is then performed through interaction with these objects.
 - This is known as **object-oriented programming (OOP)**.

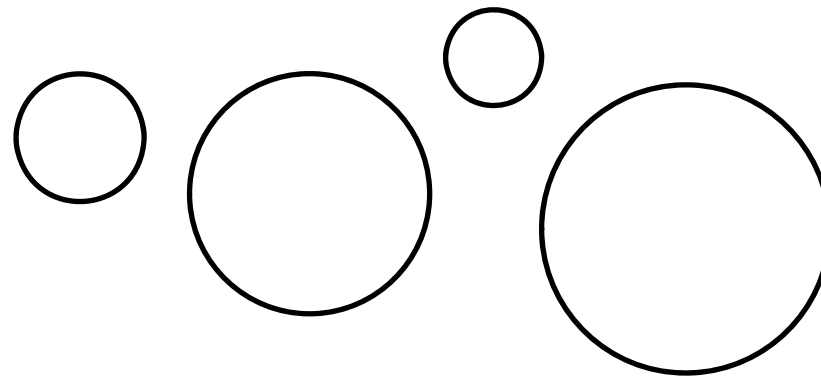
Classes and Objects

- In OO programming, a **class** describes an entity of our interest. A **class** is a template to create **objects**.
- Example:



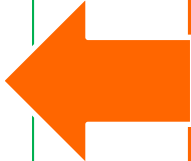
Example : Circle

- Suppose we want to define a class to describe **circles**.
 - Let's start with something simple, and add more complexity gradually.
- A circle has one attribute **radius**, which is of type **int**.
- Among other functionalities (behaviors), a circle is able to double its **radius**.



```
class Circle {  
  
    // attribute  
    private int radius;  
  
    // Constructor  
    public Circle(int r) {  
        radius = r;  
    }  
  
    // member method 1  
    public void enlarge() {  
        radius *= 2;  
    }  
  
    // member method 2  
    public int getRadius() {  
        return radius;  
    }  
}
```

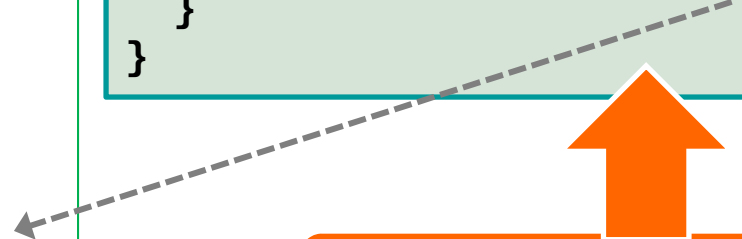
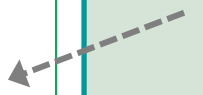
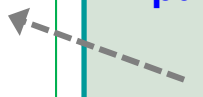
A Circle class to describe circles



```
class TestCircle { // user class  
    public static void main(String[] args) {  
        Circle cir = new Circle(4);  
        cir.enlarge();  
        System.out.println(cir.getRadius());  
    }  
}
```

8

A user class to use Circle



Now let's study the
Circle class in details.

```
class Circle {  
  
    // attribute  
    private int radius;  
  
    // Constructor  
    public Circle(int r) {  
        radius = r;  
    }  
  
    // member method 1  
    public void enlarge() {  
        radius *= 2;  
    }  
  
    // member method 2  
    public int getRadius() {  
        return radius;  
    }  
}
```

By convention, attributes are defined as **private** so that it is NOT directly accessible from user class.

```
class TestCircle { // user class  
    public static void main(String[] args) {  
        Circle cir = new Circle(4);  
        cir.radius *= 2; // syntax error  
        cir.enlarge(); // good  
        ...  
    }  
}
```

Member methods are usually defined as **public** so that they may be invoked by the user class.

Accessibility of Modifiers

- Attributes and member methods in a class can have different level of **accessibilities** (visibilities).

public

- Can be accessed from outside the class
- Usually intended for member methods only

private

- Can be assessed within this class only
- Recommended for all attributes

protected

- Can be assessed within this class, or from its child classes, or from class in the same package (**NOT covered**)

[None]

- Only accessible to classes in the same Java package (**NOT covered**)

Constructors

- A class has a specialized method called **constructor**.
 - Called **automatically** when an object is created using the keyword **new**.
 - Used for initializing the attributes of that object.
- If you never define any constructor by yourself, a default one will be inserted by the compiler during the course of compilation.
 - The default constructor takes no parameter and almost does nothing.
- However, once you define your own constructor, default constructor is no longer provided.

```
class Circle {  
  
    // attribute (also known as data member)  
    private int radius;  
  
    // Constructor  
    public Circle(int r) {  
        radius = r;  
    }  
  
    public void enlarge() {  
        radius *= 2;  
    }  
  
    public int getRadius() {  
        return radius;  
    }  
}
```

Syntax Note:

- Constructor name must be identical to the class name.
- Constructor has **NO** return type.

```
class TestCircle {  
  
    ...  
    // call constructor  
    // to create an object  
    Circle cir = new Circle(4);  
  
    cir.enlarge();  
  
    ...  
}
```

Scope of Variables

```
class Circle {
```

```
    // attribute (also known as data member)
```

```
    private int radius;
```

```
    public Circle(int r) {
```

```
        radius = r;
```

```
    }
```

```
    public void enlarge() {
```

```
        radius *= 2;
```

```
    }
```

```
    public int getRadius() {
```

```
        return radius;
```

```
    }
```

```
}
```

attributes are visible within the scope of this class.

parameters and **local variables** are visible within the method been defined.

Use **attributes** directly inside the class.

Now let's study the
TestCircle class in
details.

```
class Circle {
```

```
    // attribute
```

```
    private int radius;
```

```
    // Constructor
```

```
    public Circle (int r) {  
        radius = r;  
    }
```

```
    public void enlarge() {  
        radius *= 2;  
    }
```

```
    public int getRadius() {  
        return radius;  
    }  
}
```

Create a new object; constructor of that class will be called automatically.

```
class TestCircle {    // user class
```

```
    public static void main(String[] args) {
```

```
        Circle cir = new Circle(4);
```

```
        cir.enlarge();
```

```
        System.out.println(cir.getRadius());
```

```
    }
```

```
}
```

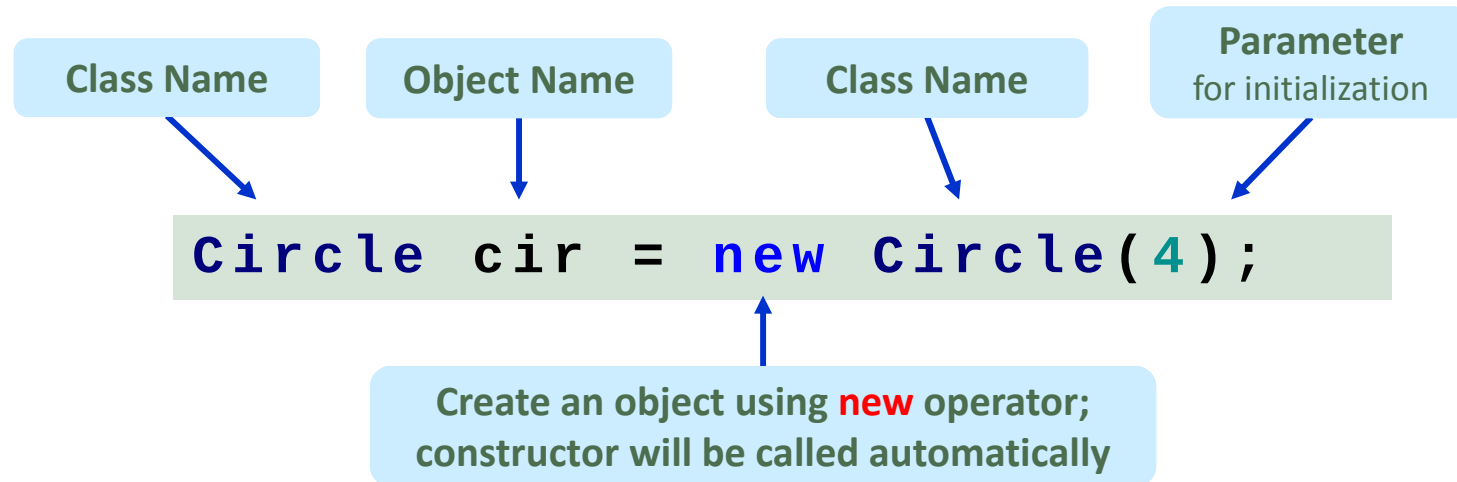
The calling object

Invoke member methods on an object to change its data or to retrieve its data.

Syntax:

object.method()

Object Declaration and Creation

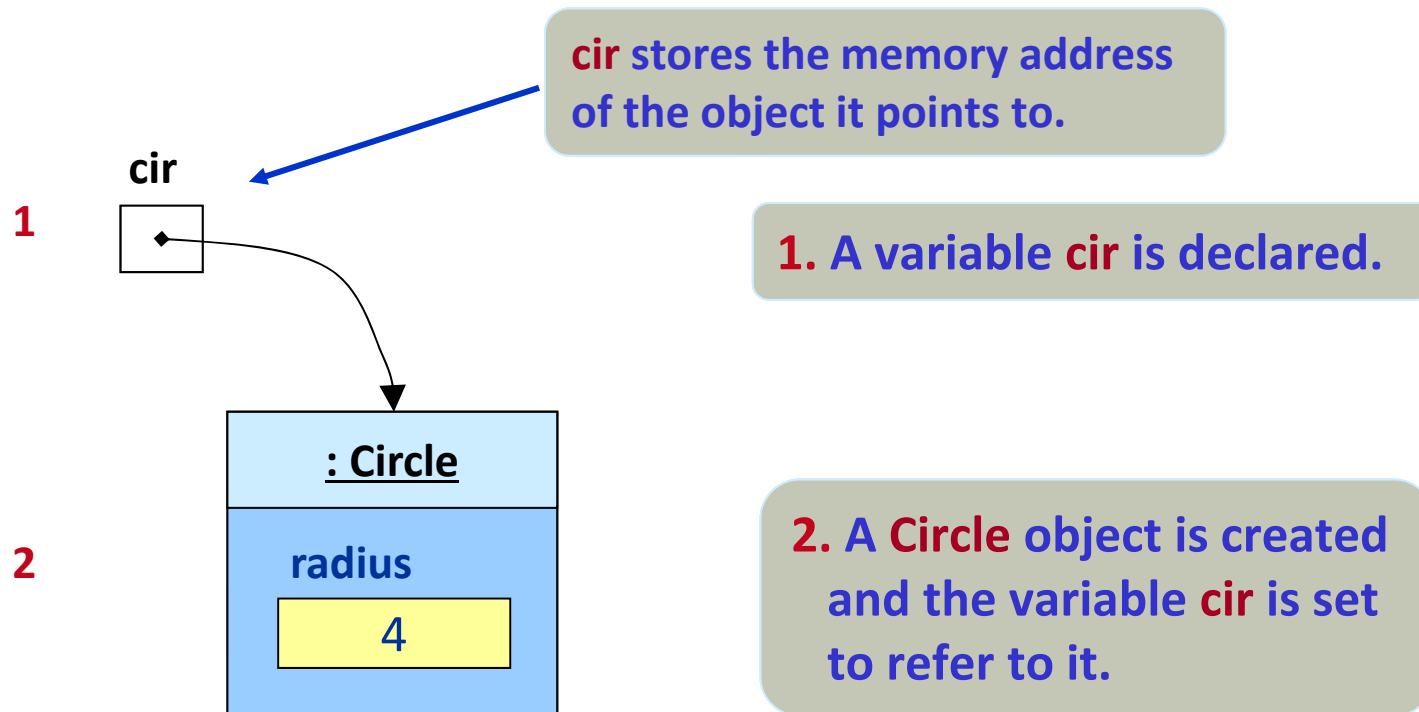


- Above statement actually does two things:
 1. Declare an **object reference (variable)** called `"cir"`.
 2. Create a **Circle object** using `"new"` and let `"cir"` points to it.
- It can be re-written in two steps as follows:

```
Circle cir; // declare a variable
cir = new Circle(4); // create an object
```

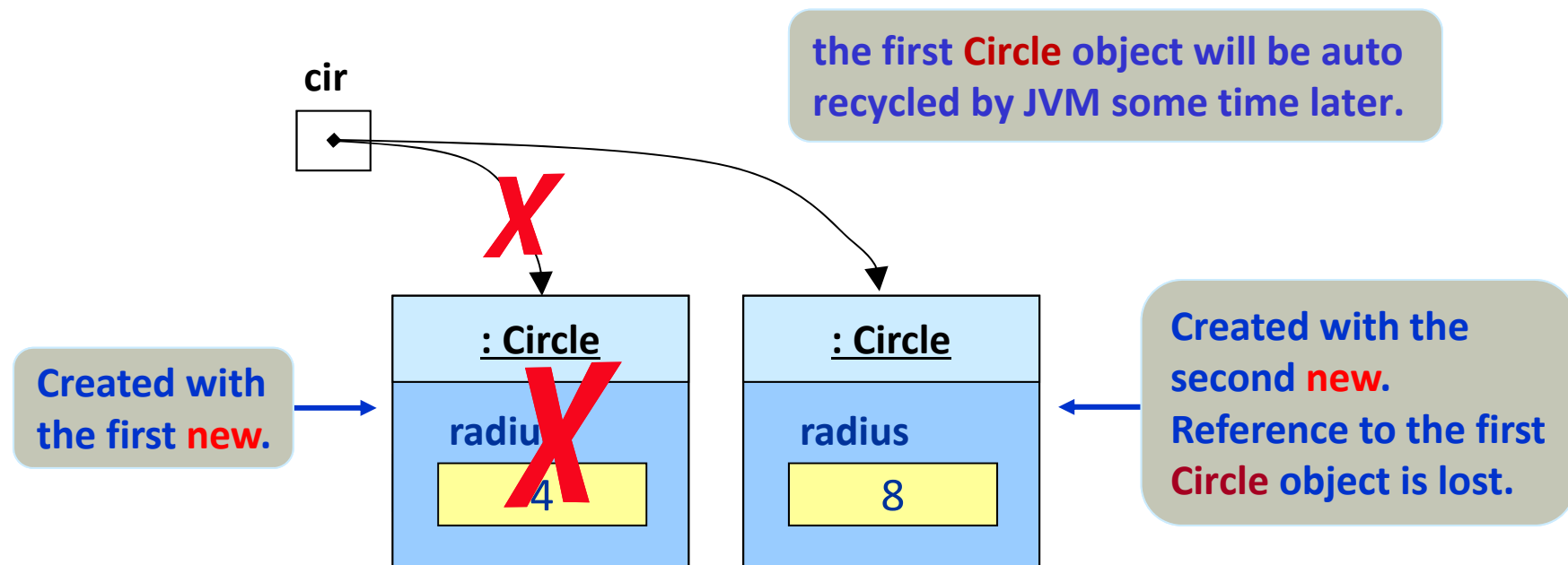
Object Reference Examples (1/4)

```
1 Circle cir; // declare an object reference
2 cir = new Circle(4); // create an object
```



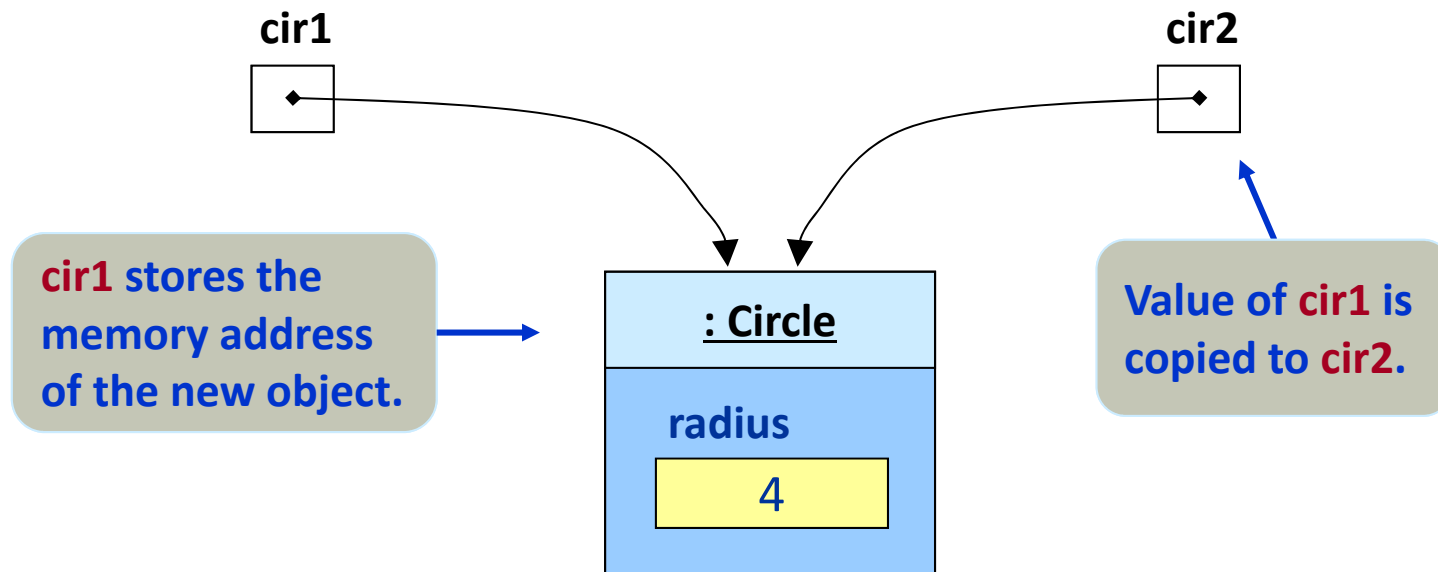
Object Reference Examples (2/4)

```
Circle cir;  
cir = new Circle(4);  
cir = new Circle(8);
```



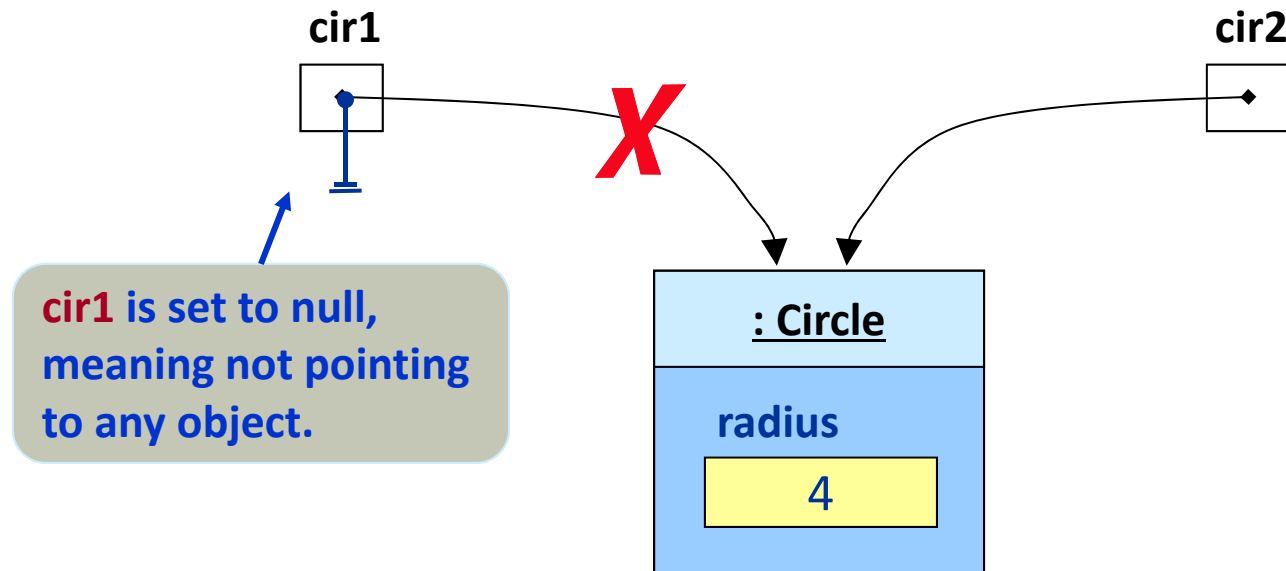
Object Reference Examples (3/4)

```
Circle cir1, cir2;  
cir1 = new Circle(4);  
cir2 = cir1;
```



Object Reference Examples (4/4)

```
Circle cir1, cir2;  
cir1 = new Circle(4);  
cir2 = cir1;  
cir1 = null;
```



To Compare with Numeric Data

```
A int num1, num2;  
B num1 = 234;  
  num2 = 37;  
C num2 = 87;
```

A. Variables are allocated in memory.

num1	234
num2	87

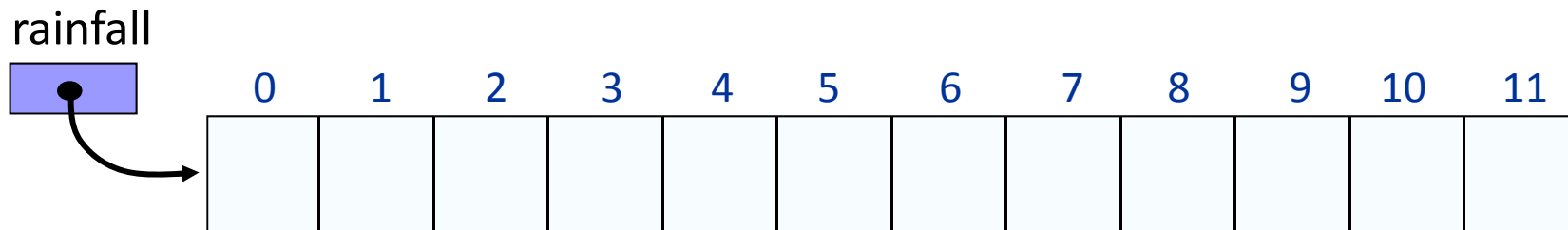
C. The value 87 overwrites the previous value 37.

B. Values are assigned to variables.

Primitive and Reference Data Types

- Numerical data is called *primitive data types*.
 - `char` is also a primitive data type.
- Objects are called *reference data types*.
 - Array and String are also reference data type.
- Example:

```
double[] rainfall = new double[12];
```



Comparing Objects

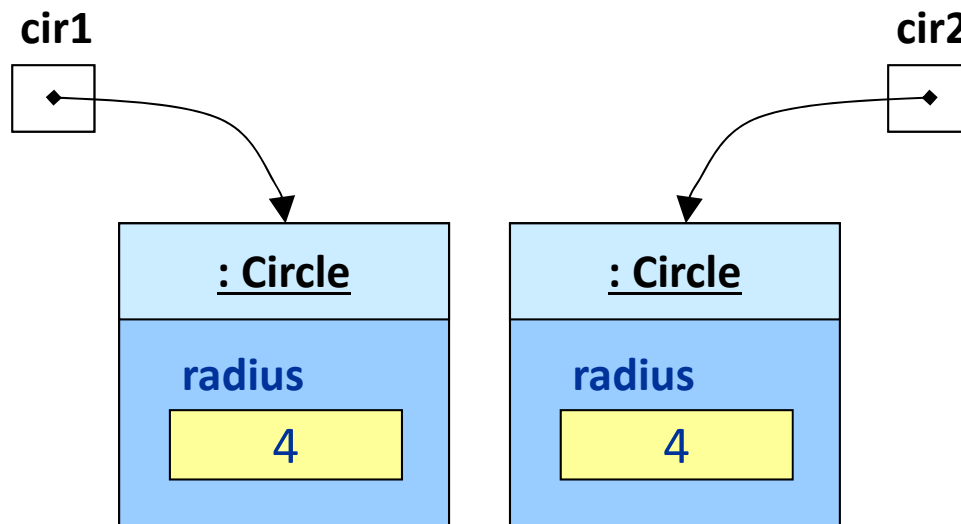


```
Circle cir1 = new Circle(4);
Circle cir2 = new Circle(4);
if (cir1 == cir2) {
    System.out.println("They are identical");
} else {
    System.out.println("They are different");
}
```

They are different

The values of **cir1** and **cir2** are the addresses of the objects they refer to.

Read OOP Unit 2 slides for the correct way to compare the contents of two objects.



cir1 == cir2 compares the addresses of the objects they refer to.

Today's Summary

Basic Object-oriented Programming I

Basic OOP concepts and features

- Class and object
- Attribute, constructor and member method
- Visibility modifier
- Object reference
- Comparing objects

